1. Problem Statement

Design a URL shortening service, similar in concept to bit.ly. The system accepts a long URL and returns a short, unique alias. When a user visits the short URL, the system redirects them to the original long URL. The service also tracks how many times each short link has been clicked.

1.1 Functional Requirements

- Generate a short, unique alias for a submitted long URL.
- Redirect any visitor who opens the short URL to the correct original URL.
- Track total click count per short link, with a rough breakdown by date.
- A given owner (user/campaign) resubmitting the same long URL receives the same short code; different owners submitting the same long URL receive distinct codes, so analytics stay attributable per campaign.

1.2 Non-Functional Requirements

- Low-latency redirects — the read path is the dominant, user-facing path.
- High availability for redirects, since broken links directly damage shared campaigns/content.
- The system should tolerate significant growth (assume up to 10x) without a redesign.

1.3 Out of Scope

- How an end consumer (marketing team, individual user) distributes or pastes the short link into emails, ads, or social posts — that workflow sits outside this system's boundary.
- Device/location/referrer-level analytics — only click counts are required for the current version.
- Link expiry — explicitly deferred; see Section 6.

2. Capacity Estimation

Back-of-the-envelope estimation drives nearly every architectural decision that follows — including which components are necessary and which would be premature optimization.

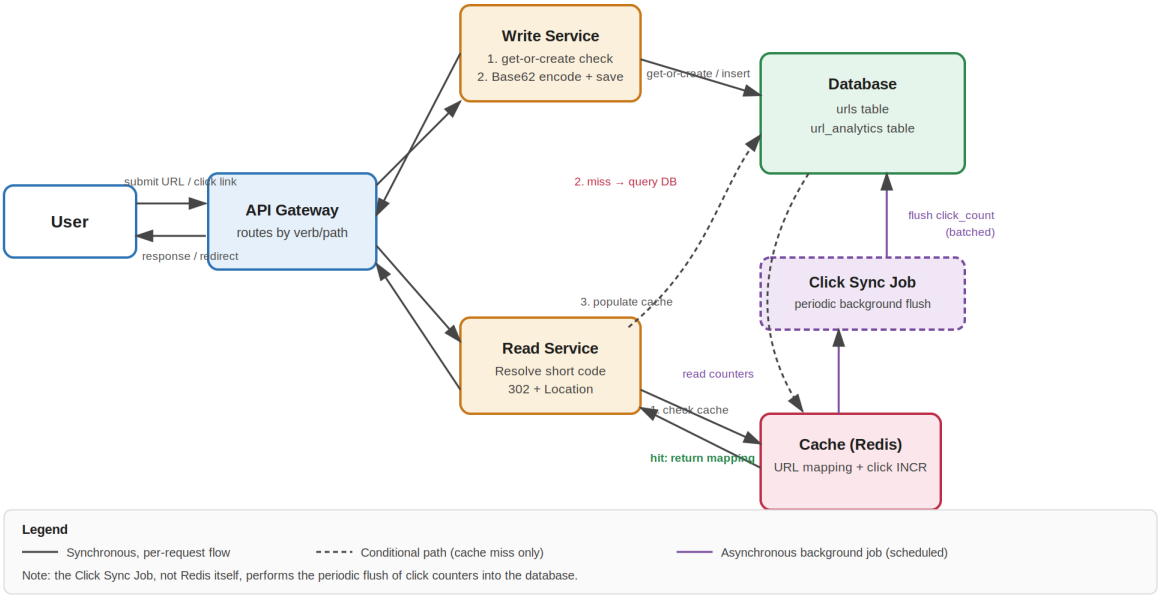
Metric	Assumption	Derived value
New short URLs created	1,000,000 / day	≈ 12 writes/sec
Redirects (clicks)	100,000,000 / day (100:1 read:write ratio)	≈ 1,200 reads/sec
Average record size	≈ 500 bytes (code, long URL, timestamps, metadata)	—
Storage over 5 years	1B records/day × 365 × 5 × 500 bytes	≈ 0.9–1 TB total

Key implication

Both write throughput (~12/sec) and total storage (~1 TB over 5 years) are comfortably within the capacity of a single, well-indexed relational database instance. Sharding is not justified by either dimension at this scale. Read traffic (~1,200/sec), combined with a power-law access pattern where a small number of links receive a disproportionate share of clicks, is the primary justification for introducing a cache.

3. High-Level Architecture

The diagram below separates the write path (URL creation) from the read path (redirect + click tracking), reflecting their very different traffic profiles and latency requirements.



3.1 Components

Component	Responsibility
API Gateway	Single entry point; routes requests to the Write or Read service based on HTTP method/path.
Write Service	Handles URL creation: get-or-create lookup, Base62 ID generation, persistence.
Read Service	Resolves a short code to a long URL via cache-first lookup and returns an HTTP redirect.
Database	System of record — URLs table and URL_analytics table (see Section 5).
Cache (Redis)	Caches short-code → long-URL mappings to protect the database from repeated lookups of hot/viral links; also holds live click counters.

Component	Responsibility
Click Sync Job	Background process that periodically reads click counters from Redis and flushes them into the database in batches.

3.2 Components Deliberately Excluded

A message queue between the Write Service and the database was considered and explicitly rejected. At ~12 writes/sec, a standard relational database handles direct writes without contention; a queue would add operational complexity (a new system to run and monitor) without solving a problem that exists at this scale.

4. API Design

4.1 Create Short URL

	Detail
Endpoint	POST /shorten
Caller	Frontend application (JavaScript)
Request body	{ "long_url": "...", "owner_id": "..." }
Success response	200 OK — { "success": true, "short_url": "...", "message": "Short URL generated successfully" }
Failure response	4xx — { "success": false, "message": "..." }
Response format rationale	JSON, because the caller is application code that must render a result in the UI (display the new link, show a copy button, show an error state).

4.2 Redirect

	Detail
Endpoint	GET /{shortCode}
Caller	A browser navigating directly — not application code
Success response	302 Found, with a Location header set to the original long URL
Failure response	404 Not Found
Response format rationale	No JSON body. A browser receiving a 3xx status with a Location header redirects automatically — this is native HTTP behaviour requiring no client-side logic.

Why 302, not 301

301 (Moved Permanently) signals browsers to cache the redirect locally and skip contacting the server on repeat visits. That would silently undercount clicks, conflicting with the analytics requirement. 302 (Found) forces every click to reach the server, at a small cost to browser-level caching efficiency — a trade-off explicitly justified by a stated requirement, not a default choice.

5. Data Model

5.1 urls table

Column	Type	Notes
id	BIGINT, PK, auto-increment	Source of the Base62-encoded short code
original_url	TEXT	—
short_url	VARCHAR	Base62-encoded id
url_owner	VARCHAR	User/campaign identifier; drives get-or-create scoping
url_category	VARCHAR	e.g. product, campaign, custom page
created_at / updated_at	TIMESTAMP	—

5.2 url_analytics table

Column	Type	Notes
url_id	BIGINT, FK → urls.id	—
click_count	BIGINT	Updated via periodic batch flush from Redis, not per-click writes

5.3 Indexing

A composite index on (original_url, url_owner) supports the get-or-create lookup performed on every write — checking whether this owner has already shortened this exact URL before generating a new code. A single composite index serves this two-column lookup far more efficiently than two separate single-column indexes, which the database could generally only use one of for this query.

6. Key Design Decisions and Trade-offs

This section documents not just what was decided, but why — and what alternatives were ruled out.

6.1 Caching Strategy: Cache-Aside

1. On a redirect request, check Redis first for the short-code → long-URL mapping.

2. On a cache hit, return immediately — no database access.
3. On a cache miss, query the database, return the result, and write it into Redis so the next request for that code is served from cache.

Without step 3, the cache would never populate itself and every request would remain a miss indefinitely — the cache-aside write-back step is what makes the pattern function, not an optional addition.

6.2 Click Analytics: Decoupled from the Redirect Path

Click counting and URL-mapping lookups are independent concerns, handled separately. Every redirect triggers a fast, atomic increment (INCR) on a Redis counter for that short code — this does not block or delay the redirect response. A separate background Click Sync Job periodically reads accumulated counters from Redis and flushes them into the url_analytics table in batches. This avoids a database write on every single click (~1,200/sec) while still capturing every click accurately, since 302 (not 301) guarantees every click reaches the server in the first place.

6.3 Short Code Generation: Base62-Encoded Auto-Increment

A monotonically increasing integer ID (native database auto-increment) is encoded into Base62 (0–9, a–z, A–Z) to produce a short, URL-safe code.

Approach	Considered	Verdict
Truncated hash (MD5/SHA-256)	Yes	Rejected — truncating a hash to a short length introduces collisions (the birthday paradox), requiring a database check-and-retry on every write.
Hash + collision handling	Yes	Viable, but adds an extra DB read and retry logic on every write for no benefit at this scale and threat model.
Base62(auto-increment)	Selected	Collision-free by construction; no extra lookup or retry needed. Codes are sequential/guessable, which is an acceptable trade-off for a marketing-link product with no sensitive destination data.

6.4 Owner-Scoped Uniqueness (Get-or-Create)

Short codes are not derived purely from the long URL. The same long URL submitted by the same owner returns the existing short code; the same long URL submitted by a different owner generates a new, distinct code. This is required because campaigns need independently attributable click analytics — two marketing teams linking to the same product page, but via different channels, must be able to see separate click counts. A single shared code for a shared destination would silently merge two distinct campaigns' data.

6.5 Deferred for Future Iteration

The following were deliberately not built into the current design, with the reasoning for deferral noted:

- **Link expiry (expire_at)** — not a stated requirement; would require coordinated logic across the API, cache TTL, and a cleanup process. Noted as a clean extension point if a future requirement calls for it.
- **Distributed ID generation** — a single database's native auto-increment is sufficient at ~12 writes/sec. If the system later requires multiple write nodes, range-based ID allocation or a Snowflake-style scheme (timestamp + node ID + sequence) would remove the need for centralized coordination.
- **Database sharding** — ruled out for both the reasons it might normally be reached for: ~1 TB over 5 years doesn't strain a single instance's storage, and ~12 writes/sec doesn't strain its throughput. Sharding solves a data-distribution problem, not an ID-generation problem, and isn't justified by either dimension here.

7. Summary

This design separates a low-volume, latency-tolerant write path from a high-volume, latency-sensitive read path, and justifies every component — including the ones left out — against measured capacity rather than assumption. The cache, the HTTP redirect semantics, the analytics flush mechanism, and the ID generation strategy were each chosen by working through a concrete failure scenario or requirement conflict, not by default convention.